

A Short Exposition on Formalization and Application of Support Vector Machines and a Paper on Residual Learning

Doruk Üstündağ
Izmir Institute of Technology

1. Understanding Support Vector Machines

1.1. Introduction to Max-Margin Classification

Firstly let us propose the ideas behind **support vector machine** classification briefly. To understand this classifier we need to understand some basic definitions in statistics and classification theory:

Definition 1 (Binary Classification). Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the input space and $\mathcal{Y} = \{0, 1\}$ the label set. Binary classification is the task of learning a function

$$h : \mathcal{X} \rightarrow \{0, 1\}$$

from a training set $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathcal{X}$ and $y_i \in \{0, 1\}$ (or equivalently $\{-1, 1\}$). Many classifiers produce a real-valued score $s(x) \in \mathbb{R}$ and apply a threshold $\tau \in \mathbb{R}$:

$$h(x) = \begin{cases} 1 & \text{if } s(x) \geq \tau, \\ 0 & \text{if } s(x) < \tau. \end{cases}$$

So to do some classification on the input space (which is namely our dataset) $\mathcal{X}$ we need to define some classifier function h :

Definition 2 (Linear Classifier). Let $\mathbf{x} \in \mathbb{R}^d$ be an input vector. A *linear classifier* is a function

$$h : \mathbb{R}^d \rightarrow \{-1, +1\}, \quad h(\mathbf{x}) = \text{sign}(w^\top \mathbf{x} + b),$$

where $w \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias term. The corresponding *decision boundary* is the hyperplane

$$\mathcal{H} = \{\mathbf{x} \in \mathbb{R}^d : w^\top \mathbf{x} + b = 0\}.$$

More generally, for an output function $f : \mathbb{R} \rightarrow \mathcal{Y}$,

$$h(\mathbf{x}) = f(w^\top \mathbf{x} + b).$$

Linear classifier and binary classification ideas can be easily understood via this geometric intuition:

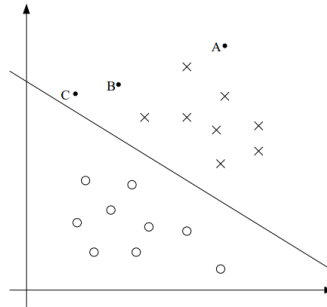


Figure 1: Linear Classification Example on a Binary Classification

We already can see that our separating hyperplane is the decision boundary given by $\theta^T x = 0$, which partitions the feature space into two half-spaces, each associated with one of the two classes. The whole idea

is: can we trust, and how much can we trust, this kind of **margin-based** classification with some **margin**? One can see our classifier can classify A easily but misclassification of C is actually the rather tricky problem which makes our idea a valid question to ask.

1.2. Constructing Idea of Margin

Optimally, maximizing the margin between first examples of both partitions is the most basic idea that comes to mind. Let us firstly formalize the idea of **margin** in a more concrete approach with two different kinds of margins:

Definition 3 (Functional Margin). Given a training example $(x^{(i)}, y^{(i)})$, the *functional margin* of (w, b) with respect to that example is defined as

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x^{(i)} + b).$$

Given a training set $\mathcal{T} = \{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$, the functional margin of (w, b) with respect to $\mathcal{T}$ is

$$\hat{\gamma} = \min_{i=1, \dots, m} \hat{\gamma}^{(i)}.$$

One must keep in mind **functional margin** scales linearly by our choice of (w, b) without changing the classifier in a meaningful way. Another *more natural and intuitive* way to approach formalization of margin idea is: **Geometric margin**. Let us build the idea of geometric margin via adding a little bit to the intuition we gave before about margins:

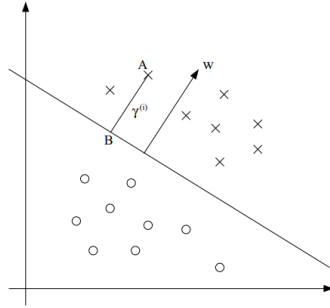


Figure 2: Geometric Margin Construction on the Linear Classifier

The decision boundary satisfies $w^T x + b = 0$, and w is orthogonal to it. For a positive example $x^{(i)}$ at point A , the projected point B on the boundary is

$$B = x^{(i)} - \gamma^{(i)} \frac{w}{\|w\|}.$$

Since B lies on the boundary, substituting gives

$$w^T \left(x^{(i)} - \gamma^{(i)} \frac{w}{\|w\|} \right) + b = 0.$$

Solving for $\gamma^{(i)}$ yields:

$$\gamma^{(i)} = \frac{w^T x^{(i)} + b}{\|w\|} = \left(\frac{w}{\|w\|} \right)^T x^{(i)} + \frac{b}{\|w\|}.$$

Thus we can define what is geometric margin from this construction directly:

Definition 4 (Geometric Margin). The *geometric margin* of (w, b) with respect to a training example $(x^{(i)}, y^{(i)})$ is defined as

$$\gamma^{(i)} = y^{(i)} \left(\left(\frac{w}{\|w\|} \right)^T x^{(i)} + \frac{b}{\|w\|} \right).$$

Given a training set $\mathcal{T}$, the geometric margin of (w, b) with respect to $\mathcal{T}$ is

$$\gamma = \min_{i=1, \dots, m} \gamma^{(i)}.$$

Using this remark we can get the connection between **geometric** and **functional** margin:

Remark 1. Note that if $\|w\| = 1$, the functional margin equals the geometric margin, and the geometric margin is invariant to rescaling of (w, b) .

As we understood before it is good to say that a larger geometric margin gives us a *more confident* classification on our training set. For our approach on finding a max-margin classifier we must assume that our training examples are linearly separable. So our problem reduces to:

Question 1. *How can we find the hyperplane that achieves the maximum geometric margin?*

For this question we can easily propose an optimization problem as

$$\begin{aligned} \max_{\gamma, w, b} \quad & \gamma \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq \gamma, \quad i = 1, \dots, m \\ & \|w\| = 1. \end{aligned}$$

We aim to maximize γ subject to the constraint that each training example has functional margin at least γ . Imposing $\|w\| = 1$ ensures that the functional margin coincides with the geometric margin, and thus all geometric margins are also at least γ . Consequently, solving this optimization problem yields (w, b) that achieves the maximum possible geometric margin with respect to the training set. With a few steps of *playing with constraints and scalings* we can get the convex problem:

$$\begin{aligned} \max_{\gamma, w, b} \quad & \hat{\gamma} \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq \hat{\gamma}, \quad i = 1, \dots, m, \end{aligned}$$

which is actually an answer to our question. We can easily solve this kind of convex quadratic optimization problem using **Quadratic Programming** solvers.

1.3. Dual Problem and Support Vectors

For an optimization problem with some *smooth* constraints it is quite natural to think about the dual problem of the main problem via **Lagrange Duality**. Because of length constraints one can consult [3] and [6] for a lengthy exposition on the construction of the dual problem. We will be directly stating the dual form of the problem and the idea of the celebrated **support vectors**: via using the constraints given in the proposed optimization problem we can write the function $g_i(w)$:

$$g_i(w) = -y^{(i)}(w^T x^{(i)} + b) + 1 \leq 0,$$

as we have one such constraining function g_i for each training sample. From some condition which is important for constructing our dual problem we can get that we get positive contribution only from training examples whose functional margin is *exactly* equal to one, that is to say, those corresponding to active constraints $g_i(w) = 0$. Now by these training examples we can define what a **support vector** is:

Definition 5 (Support Vectors). The **support vectors** are the training examples $(x^{(i)}, y^{(i)})$ for which the corresponding Lagrange multiplier $\alpha_i^* > 0$ at the optimal solution. By some optimal condition,

$$\alpha_i > 0 \implies g_i(w) = 0 \implies y^{(i)}(w^T x^{(i)} + b) = 1,$$

i.e., support vectors are precisely those points lying on the *margin hyperplanes*

$$\mathcal{H}_{\pm} = \{x \in \mathbb{R}^d : w^T x + b = \pm 1\},$$

at geometric distance $\gamma = 1/\|w\|$ from the separating hyperplane $\mathcal{H}_0 = \{x : w^T x + b = 0\}$. All other training examples have $\alpha_i = 0$ and do not contribute to the solution.

We can continue with a short construction of the dual problem. We consider our optimization problem in its *primal* form.

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1, \dots, m.$$

To handle the inequality constraints, we introduce nonnegative Lagrange multipliers $\alpha_i \geq 0$, one for each training example, and form the Lagrangian

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^m \alpha_i [y^{(i)}(w^T x^{(i)} + b) - 1].$$

The dual problem is obtained by minimizing this Lagrangian with respect to the primal variables w and b for fixed α . Taking the gradient of $\mathcal{L}$ with respect to w and setting it to zero yields

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} = 0,$$

which implies

$$w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}.$$

Similarly, differentiating with respect to b gives

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = \sum_{i=1}^m \alpha_i y^{(i)} = 0.$$

Thus, at any optimum, the weight vector is a linear combination of the training points, and the multipliers must satisfy a single linear equality constraint.

Substituting the expression for w back into the Lagrangian, we obtain

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)} - b \sum_{i=1}^m \alpha_i y^{(i)}.$$

Using the condition $\sum_{i=1}^m \alpha_i y^{(i)} = 0$, the last term vanishes and the Lagrangian, evaluated at its minimizer in w and b , reduces to

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)}.$$

This expression defines the dual objective function

$$W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle,$$

where $\langle x^{(i)}, x^{(j)} \rangle$ denotes the standard inner product in the input space. The dual optimization problem is then given by

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\ \text{subject to} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m, \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0. \end{aligned}$$

Under the usual regularity conditions for this convex problem we can arrive at our dual problem. Once the dual problem has been solved and the maximizing multipliers α_i^* have been found, we can recover the optimal primal parameters. The optimal weight vector is

$$w^* = \sum_{i=1}^m \alpha_i^* y^{(i)} x^{(i)}.$$

The optimal intercept b^* can be obtained by enforcing that the separating hyperplane lies halfway between the closest points of the two classes in the projection induced by w^* . One convenient formula is

$$b^* = - \frac{\max_{i: y^{(i)} = -1} (w^*)^T x^{(i)} + \min_{i: y^{(i)} = 1} (w^*)^T x^{(i)}}{2}.$$

After understanding the dual problem, the prediction at a new point x reduces to:

$$w^T x + b = \left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} \right)^T x + b = \sum_{i=1}^m \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b.$$

We predict $y = 1$ if and only if $w^T x + b > 0$. Since $\alpha_i = 0$ for all non-support vectors, only inner products with support vectors need to be computed (this classification puts emphasis on support vectors because of this). Thus using this property of the problem we can use something called *kernel trick* and then we can call this classification model algorithm a **support vector machine**.

1.4. Kernel Trick

Rather than applying SVMs on raw input attributes x , we may learn using a **feature mapping** $\phi : \mathbb{R}^d \rightarrow \mathcal{F}$. For working with these feature mappings we must introduce something called **kernel function**:

Definition 6 (Kernel Function). Given a feature mapping ϕ , the corresponding **kernel** is defined as:

$$K(x, z) = \phi(x)^T \phi(z).$$

Replacing $\langle x, z \rangle$ with $K(x, z)$ everywhere in the algorithm allows implicit computation in $\mathcal{F}$ without ever explicitly evaluating $\phi(x)$.

Via this important definition we can actually understand what is the “kernel trick”:

Proposition 1 (Kernel Trick). *If a learning algorithm can be written entirely in terms of inner products $\langle x, z \rangle$ between input vectors, then replacing each inner product with a kernel $K(x, z)$ allows the algorithm to operate implicitly in the high-dimensional feature space corresponding to K :*

$$\langle x, z \rangle \longmapsto K(x, z) = \phi(x)^T \phi(z).$$

From our prediction rule given before we can easily change our inner product into a kernel:

$$w^T x + b = \sum_{i=1}^m \alpha_i y^{(i)} K(x^{(i)}, x) + b.$$

Let us give an example of a simple kernel:

Example 1 (Polynomial Kernel). Let $x, z \in \mathbb{R}^n$ and consider $K(x, z) = (x^T z)^2$:

$$K(x, z) = \left(\sum_{i=1}^n x_i z_i \right)^2 = \sum_{i,j=1}^n (x_i x_j) (z_i z_j) = \phi(x)^T \phi(z),$$

where $\phi(x) = [\dots, x_i x_j, \dots]^T$ contains all degree-2 monomials.

This kernel gives us explicit computational advantages:

- Computing $\phi(x)$ explicitly requires $O(n^2)$ time.
- Computing $K(x, z) = (x^T z)^2$ requires only $O(n)$ time.
- More generally, $K(x, z) = (x^T z + c)^d$ corresponds to a feature space of all monomials up to degree d , of dimension $\binom{n+d}{d}$, yet costs only $O(n)$ to evaluate.

The kernel trick actually gives us more than just *computational efficiency*; in some cases it helps us to separate non-linearly separable data in higher (possibly infinite) dimensions by embedding the data as feature vectors and working on that space with our **kernels**.

2. Application of Support Vector Machine Classification on Breast Cancer Data

2.1. UCI Wisconsin Breast Cancer Dataset

The experiments in this work use the Breast Cancer Wisconsin (Diagnostic) dataset, originally constructed at the University of Wisconsin Hospitals by Wolberg et al. and distributed via the UCI Machine Learning Repository (doi:10.24432/C5DW2B).¹ The dataset contains 569 observations of breast masses obtained by fine needle aspirate (FNA) biopsy. Each observation corresponds to a digitized image of a FNA sample, from which quantitative features of cell nuclei are extracted. For each case, a binary diagnosis is provided, indicating whether the mass is malignant or benign.

Structure. Each sample is described by 30 real-valued input features and a binary class label. The class takes two values: *malignant* (M) and *benign* (B). In the standard version used by `scikit-learn` (via `load_breast_cancer`), the data set has the following characteristics:

- Number of instances: 569 (212 malignant, 357 benign),
- Number of features: 30 (all continuous, non-negative),
- Number of classes: 2 (binary diagnosis),
- Missing values: none reported.

Feature construction. For each cell nucleus in the image, ten basic morphometric measurements are computed:

1. radius (mean distance from center to points on the perimeter),
2. texture (standard deviation of gray-scale values),
3. perimeter,
4. area,
5. smoothness (local variation in radius lengths),
6. compactness ($\text{perimeter}^2/\text{area} - 1.0$),
7. concavity (severity of concave portions of the contour),
8. concave points (number of concave portions of the contour),
9. symmetry,
10. fractal dimension (“coastline approximation” of the contour).

For each of these ten quantities, three aggregate statistics are computed over all nuclei in the image: the mean value, the standard error, and the “worst” (maximum) value. This yields $10 \times 3 = 30$ features per observation. This dataset is a standard benchmark for binary classification in medical machine learning. It is often used to evaluate algorithms like support vector machines, typically after feature standardization due to the different numerical scales of the predictors.

2.2. Experimental Setup: SVM Learning Curves on the Breast Cancer Dataset

In this project, we study the effect of training set size and kernel choice on the performance of support vector machines (SVMs) for binary classification on the Breast Cancer Wisconsin (Diagnostic) dataset, accessed via `sklearn.datasets.load_breast_cancer`. [8] The code is implemented in Python using `scikit-learn` for modeling, NumPy and pandas for data handling, and matplotlib/seaborn for visualization. For each specified training fraction and for multiple random iterations, we perform hyperparameter tuning with cross-validation, evaluate the best model on a held-out test set, and then aggregate the results into learning curves with respect to accuracy, F1-score, and Matthews correlation coefficient (MCC).

¹UCI Machine Learning Repository, *Breast Cancer Wisconsin (Diagnostic)*.

Configuration. The main experimental configuration (train–test split, training fractions, number of iterations, cross-validation, and scoring metric) is defined in a single dictionary as shown in Listing 1. We fix the random seed for reproducibility and use MCC as the primary tuning metric due to the class-imbalance properties of the dataset.

Listing 1: Configuration for SVM learning-curve experiments on the Breast Cancer Wisconsin (Diagnostic) dataset

```
CONFIG = {
    "name": "Breast Cancer Learning Curve with Tuning (SVM Kernels)",
    "test_size": 0.10,
    "train_fractions": [0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70],
    "n_iterations": 10,
    "random_seed": 42,
    "cv_folds": 5,
    "scoring": "matthews_corrcoef",
    "output_dir": "svm_breast_cancer_results",
}
```

SVM pipelines and hyperparameters. We consider a single SVM model family but vary the kernel type and associated hyperparameters. As shown in Listing 2, we build a `Pipeline` combining a feature scaler (either `StandardScaler` or `MinMaxScaler`) with `sklearn.svm.SVC`. For each kernel (`linear`, `rbf`, and `poly`), we specify a grid over the regularization parameter C , the kernel parameters (γ , degree, and offset), and the optional `class_weight` setting. This grid is passed to `GridSearchCV` for exhaustive hyperparameter search.

Listing 2: SVM pipeline and hyperparameter grids for linear, RBF, and polynomial kernels

```
MODEL_SPECS = {
    "SVM": {
        "pipeline": Pipeline([
            ("scaler", StandardScaler()),
            ("model", SVC()),
        ]),
        "param_grid": [
            {
                "scaler": [StandardScaler(), MinMaxScaler()],
                "model__kernel": ["rbf"],
                "model__C": [0.1, 1, 10, 100],
                "model__gamma": ["scale", "auto", 0.01, 0.001],
                "model__class_weight": [None, "balanced"],
            },
            {
                "scaler": [StandardScaler(), MinMaxScaler()],
                "model__kernel": ["linear"],
                "model__C": [0.1, 1, 10, 100],
                "model__class_weight": [None, "balanced"],
            },
            {
                "scaler": [StandardScaler(), MinMaxScaler()],
                "model__kernel": ["poly"],
                "model__C": [0.1, 1, 10, 100],
                "model__degree": [2, 3, 4],
                "model__gamma": ["scale", "auto", 0.01, 0.001],
                "model__coef0": [0.0, 0.5, 1.0],
                "model__class_weight": [None, "balanced"],
            },
        ],
    },
}
```

Dataset loading and preprocessing. The dataset is loaded using the `load_breast_cancer` utility, which returns the feature matrix, target labels, and metadata. [8] For convenience, we attach a simple

textual description to each sample, which is used later for identifying and inspecting misclassified cases (e.g. false negatives). The corresponding code is given in Listing 3.

Listing 3: Loading the Breast Cancer Wisconsin (Diagnostic) dataset from `scikit-learn`

```
def load_dataset():
    data = load_breast_cancer()
    X = data.data
    y = data.target
    descriptions = [f"sample_{i}" for i in range(len(y))]
    return X, y, descriptions
```

Hyperparameter tuning and evaluation. For each train/test split and training fraction, we perform stratified subsampling on the training set to obtain the desired fraction, then conduct hyperparameter tuning using stratified k -fold cross-validation. The best pipeline (scaler + SVM with chosen kernel and hyperparameters) is refit on the subsampled training data and evaluated on the held-out test set. We compute standard classification metrics (accuracy, precision, recall, F1-score) along with MCC and the confusion matrix. A compact version of the core tuning and evaluation routine is shown in Listing 4.

Listing 4: Hyperparameter tuning with cross-validation and evaluation on the held-out test set

```
def tune_and_evaluate(X_sub, y_sub, X_test, y_test,
                      model_name, spec, config, seed):
    cv = StratifiedKFold(
        n_splits=config["cv_folds"],
        shuffle=True,
        random_state=seed,
    )
    gs = GridSearchCV(
        estimator=clone(spec["pipeline"]),
        param_grid=spec["param_grid"],
        scoring=config["scoring"],
        cv=cv,
        n_jobs=-1,
        refit=True,
    )
    start = time.time()
    gs.fit(X_sub, y_sub)
    tune_time = time.time() - start

    best_model = gs.best_estimator_
    y_pred = best_model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    mcc = matthews_corrcoef(y_test, y_pred)

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    cv_acc = cross_validate(
        best_model, X_sub, y_sub, cv=cv,
        scoring="accuracy", n_jobs=-1,
        return_train_score=False,
    )["test_score"]

    return {
        "model": model_name,
        "kernel": gs.best_params_["model__kernel"],
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
```

```

    "mcc": mcc,
    "cv_accuracy_mean": cv_acc.mean(),
    "cv_accuracy_std": cv_acc.std(),
    "best_cv_score": gs.best_score_,
    "best_params": gs.best_params_,
    "train_time_seconds": tune_time,
    "cm_TN": tn, "cm_FP": fp,
    "cm_FN": fn, "cm_TP": tp,
}

```

The full experiment loops over all specified training fractions and iterations, collects the resulting metrics in a `pandas` data frame, writes them to CSV files, and finally generates learning-curve plots (accuracy, F1, MCC as a function of training fraction) for each kernel type.

2.3. Results

For this SVM structure and our dataset we obtained the following results:

Experiment 1. *Wisconsin Breast Cancer Dataset, SVM (Learning Curve), Small Dataset*

SVM (Linear) — Best Hyperparameters

Training Fraction	Kernel	C	Class Weight	Scaler
10%	linear	1	None	Standard
30%	linear	1	balanced	Standard
40%	linear	1	balanced	MinMax
50%	linear	0.1	balanced	Standard
70%	linear	0.1	None	Standard

Experiment results with best parameters:

Model	Train Fraction	Accuracy	Precision	Recall
SVM_Linear	10%	0.9649	0.9474	1.0000
SVM_Linear	30%	0.9825	0.9730	1.0000
SVM_Linear	40%	0.9825	0.9730	1.0000
SVM_Linear	50%	0.9825	0.9730	1.0000
SVM_Linear	70%	0.9649	0.9474	1.0000

Model	Train Fraction	F1	MCC	Train Time (s)
SVM_Linear	10%	0.9730	0.9258	1.115
SVM_Linear	30%	0.9863	0.9626	1.203
SVM_Linear	40%	0.9863	0.9626	1.260
SVM_Linear	50%	0.9863	0.9626	1.302
SVM_Linear	70%	0.9730	0.9258	1.817

Model	Train Fraction	CV Accuracy
SVM_Linear	10%	1.0000 ± 0.0000
SVM_Linear	30%	0.9802 ± 0.0162
SVM_Linear	40%	0.9805 ± 0.0239
SVM_Linear	50%	0.9726 ± 0.0200
SVM_Linear	70%	0.9832 ± 0.0105

SVM (Poly) — Best Hyperparameters

Train Fraction	Kernel	C	γ	Degree	Coef0	Class Weight	Scaler
10%	poly	10	0.01	3	1.0	balanced	MinMax
20%	poly	0.1	scale	3	0.5	balanced	MinMax
30%	poly	1	0.01	2	1.0	balanced	Standard
40%	poly	100	auto	2	1.0	balanced	MinMax
50%	poly	10	scale	2	1.0	balanced	MinMax
60%	poly	1	0.01	4	1.0	balanced	Standard
70%	poly	100	auto	4	1.0	balanced	MinMax

Experiment results with best parameters:

Model	Train Fraction	Accuracy	Precision	Recall
SVM_Poly	10%	0.9649	1.0000	0.9444
SVM_Poly	20%	0.9649	0.9474	1.0000
SVM_Poly	30%	0.9825	0.9730	1.0000
SVM_Poly	40%	1.0000	1.0000	1.0000
SVM_Poly	50%	1.0000	1.0000	1.0000
SVM_Poly	60%	1.0000	1.0000	1.0000
SVM_Poly	70%	1.0000	1.0000	1.0000

Model	Train Fraction	F1	MCC	Train Time (s)
SVM_Poly	10%	0.9714	0.9286	1.164
SVM_Poly	20%	0.9730	0.9258	1.157
SVM_Poly	30%	0.9863	0.9626	1.182
SVM_Poly	40%	1.0000	1.0000	1.225
SVM_Poly	50%	1.0000	1.0000	1.318
SVM_Poly	60%	1.0000	1.0000	1.452
SVM_Poly	70%	1.0000	1.0000	1.803

Model	Train Fraction	CV Accuracy
SVM_Poly	10%	0.9618 $\pm$ 0.0469
SVM_Poly	20%	0.9810 $\pm$ 0.0381
SVM_Poly	30%	0.9742 $\pm$ 0.0316
SVM_Poly	40%	0.9755 $\pm$ 0.0154
SVM_Poly	50%	0.9766 $\pm$ 0.0145
SVM_Poly	60%	0.9837 $\pm$ 0.0102
SVM_Poly	70%	0.9777 $\pm$ 0.0068

SVM (RBF) — Best Hyperparameters

Train Fraction	Kernel	C	γ	Class Weight	Scaler
10%	rbf	1	0.01	None	Standard
20%	rbf	1	scale	None	MinMax
30%	rbf	10	auto	None	MinMax
40%	rbf	100	0.01	None	MinMax
50%	rbf	1	scale	balanced	MinMax
60%	rbf	100	auto	None	MinMax
70%	rbf	10	auto	balanced	MinMax

Experiment results with best parameters:

Model	Train Fraction	Accuracy	Precision	Recall
SVM_RBF	10%	0.9649	0.9474	1.0000
SVM_RBF	20%	0.9825	0.9730	1.0000
SVM_RBF	30%	0.9825	0.9730	1.0000
SVM_RBF	40%	0.9825	0.9730	1.0000
SVM_RBF	50%	1.0000	1.0000	1.0000
SVM_RBF	60%	1.0000	1.0000	1.0000
SVM_RBF	70%	0.9825	1.0000	0.9722

Model	Train Fraction	F1	MCC	Train Time (s)
SVM_RBF	10%	0.9730	0.9258	5.473
SVM_RBF	20%	0.9863	0.9626	1.229
SVM_RBF	30%	0.9863	0.9626	1.210
SVM_RBF	40%	0.9863	0.9626	1.235
SVM_RBF	50%	1.0000	1.0000	1.549
SVM_RBF	60%	1.0000	1.0000	1.523
SVM_RBF	70%	0.9859	0.9633	1.479

Model	Train Fraction	CV Accuracy
SVM_RBF	10%	0.9600 ± 0.0490
SVM_RBF	20%	0.9805 ± 0.0239
SVM_RBF	30%	0.9867 ± 0.0267
SVM_RBF	40%	0.9854 ± 0.0195
SVM_RBF	50%	0.9962 ± 0.0077
SVM_RBF	60%	0.9837 ± 0.0147
SVM_RBF	70%	0.9749 ± 0.0104

From these results we actually see that SVM with some kernels can directly separate malignant and benign tumor data.

3. Residual Learning

3.1. Introduction

Firstly, before getting on with the paper of He et al. named “Deep Residual Learning for Image Recognition” [4], we must understand the underlying theory and structure of deep neural networks, convolutional networks and the problem of image recognition. Let us state what a neural network is mathematically:

Definition 7 (Neural Network). A **(feedforward) neural network** with $L \in \mathbb{N}$ layers is a parametric family of functions

$$f_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L},$$

obtained by composing affine maps with nonlinearities:

$$\begin{aligned} h_0(x) &= x, \\ h_\ell(x) &= \sigma_\ell(W_\ell h_{\ell-1}(x) + b_\ell), \quad \ell = 1, \dots, L-1, \\ f_\theta(x) &= W_L h_{L-1}(x) + b_L, \end{aligned}$$

where $\theta = \{(W_\ell, b_\ell)\}_{\ell=1}^L$ are the trainable parameters, $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $b_\ell \in \mathbb{R}^{d_\ell}$, and each $\sigma_\ell : \mathbb{R}^{d_\ell} \rightarrow \mathbb{R}^{d_\ell}$ acts componentwise (e.g. ReLU, sigmoid, tanh). The network is typically trained by minimizing a loss function $L(f_\theta(x), y)$ over a dataset.

The task of computer vision is a topic which researchers have meddled with since the 1960s with the ideas of artificial intelligence and human mimicry. This was actually one of the most “hot topic” questions in AI

research: can a machine understand what it sees? Some techniques from **deep learning** and **convolutional networks** from more recent times helped us to build systems for image recognition and analysis more efficiently and powerfully. One of the important “inventions” is the **residual connection network**. Before going onto the theory and application examples of residual networks let us define what a convolutional neural network is:

Definition 8 (Convolutional Neural Networks). A **convolutional neural network (CNN)** is a neural network in which some layers are *convolutional* rather than fully connected. A 2D convolutional layer maps an input tensor $x \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ to an output $y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$ via

$$y_{i,j,c} = \sigma \left(\sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c'=0}^{C_{\text{in}}-1} k_{u,v,c',c} x_{i+u,j+v,c'} + b_c \right),$$

where $k \in \mathbb{R}^{K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}}$ is the filter tensor, b_c is a bias for each output channel c , and σ is a nonlinearity. A CNN is obtained by composing such convolutional layers (optionally with pooling and normalization) with fully connected layers as in the general neural network definition.

Early convolutional neural networks demonstrated promising results on relatively simple image recognition tasks; however, they struggled to capture complex and high-level visual patterns. Researchers asked an important question:

Question 2. *Do networks learn better with more layers?*

A natural response was to try and increase network depth, as deeper architectures allow models to learn increasingly abstract and hierarchical feature representations. Yet, in practice, simply stacking more layers does not guarantee improved performance — and often leads to **degraded** results. This observation prompted researchers to investigate a fundamental question: *Why do deeper networks become harder to train?* The answer is the **degradation problem**.

3.2. The Degradation Problem

To formalize the degradation problem, consider a plain network of depth L and a strictly deeper plain network of depth $L + k$. If the additional k layers could learn identity mappings, the deeper network would achieve training error no greater than its shallower counterpart [4]. However, empirical evidence in He et al. shows that this is not the case: the 56-layer plain network achieves *higher* training error than the 20-layer plain network throughout the entire training procedure [4]. This is striking because the optimization difficulty is not attributable to vanishing or exploding gradients — plain networks trained with Batch Normalization still exhibit healthy gradient norms — yet they converge at exponentially slower rates as depth increases.

Remark 2. The degradation problem is **not** a consequence of overfitting. Higher training error (not just validation error) is observed for the deeper plain network, which rules out excess model capacity as the cause. Instead, the issue lies in the difficulty solvers face when trying to approximate identity mappings through a stack of nonlinear layers.

3.3. Residual Learning Framework

He et al. address this by reformulating what each block of layers is asked to learn [4]. Rather than demanding that a stack of layers directly approximate a desired underlying mapping $\mathcal{H}(x)$, the layers are instead asked to approximate the **residual** of that mapping with respect to the identity.

Definition 9 (Residual Block). Let $x \in \mathbb{R}^d$ be the input to a building block. A **residual block** is a mapping

$$y = \mathcal{F}(x, \{W_i\}) + x,$$

where $\mathcal{F}(x, \{W_i\})$ is the residual function to be learned by the stacked layers (e.g., $\mathcal{F} = W_2 \sigma(W_1 x)$ for a two-layer block with ReLU σ), and the term $+x$ is realized via a **shortcut connection** that performs the identity mapping. When the dimensions of x and $\mathcal{F}(x)$ do not match, a linear projection W_s is used:

$$y = \mathcal{F}(x, \{W_i\}) + W_s x.$$

Identity shortcuts introduce neither extra parameters nor additional computational complexity.

Remark 3. The reformulation $\mathcal{H}(x) = \mathcal{F}(x) + x$ is motivated by the following observation: if the optimal mapping for a block happens to be the identity, it is far easier for the solver to drive $\mathcal{F}(x) \rightarrow 0$ (a zero mapping) than to learn the identity $\mathcal{H}(x) = x$ from scratch through a composition of nonlinear layers.

3.4. Gradient Flow Through Shortcut Connections

A key mathematical benefit of residual connections concerns backpropagation. For a residual block $y = \mathcal{F}(x) + x$, the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \left(\frac{\partial \mathcal{F}(x)}{\partial x} + 1 \right).$$

The identity term $+1$ ensures that gradients can flow directly back through the shortcut path even when $\partial \mathcal{F} / \partial x$ is very small, thereby directly solving the vanishing gradient problem in deep architectures [4].

Remark 4. This is precisely why residual connections became a foundational design principle in modern deep learning: they provide a **gradient highway** that bypasses the weight layers entirely, preventing the exponential decay of gradient signal that would otherwise cripple very deep plain networks.

3.5. Network Architectures

He et al. construct a family of residual networks (ResNets) of increasing depth — 18, 34, 50, 101, and 152 layers — for the ImageNet benchmark [4]. The shallower variants (ResNet-18 and ResNet-34) use standard two-layer building blocks of the form $\mathcal{F} = W_2 \sigma(W_1 x)$. For deeper variants (ResNet-50, 101, 152), a **bottleneck** design is employed to keep computational cost manageable.

Definition 10 (Bottleneck Block). A **bottleneck residual block** replaces the two-layer block with a three-layer composition:

$$\mathcal{F}(x) = W_3 \sigma(W_2 \sigma(W_1 x)),$$

where the first 1×1 convolution reduces the channel dimension, the 3×3 convolution performs feature extraction in the lower-dimensional space, and the final 1×1 convolution restores the original dimension. This design reduces the time complexity of the block while preserving its representational capacity.

Remark 5. Remarkably, the 152-layer ResNet has *lower* computational complexity (11.3 billion FLOPs) than VGG-16 and VGG-19 (15.3 and 19.6 billion FLOPs, respectively), demonstrating that depth and efficiency are not in conflict when residual connections are used [4].

3.6. Experimental Results

The experimental results directly validate the theoretical motivation. On the ImageNet validation set with 10-crop testing, plain-34 achieves 28.54% top-1 error, while ResNet-34 reduces this to 25.03% — a 3.5% absolute improvement gained solely by adding shortcut connections, with no additional parameters [4]. Going deeper, ResNet-152 achieves a single-model top-5 validation error of 4.49%, surpassing all prior ensemble results. An ensemble of ResNets achieved 3.57% top-5 error on the ImageNet test set, winning 1st place in ILSVRC 2015 [4].

Remark 6. The degradation problem is entirely absent in residual networks: for ResNets, a 34-layer model is strictly better than an 18-layer model, and the accuracy gap continues to grow as depth increases to 50, 101, and 152 layers. For plain networks, the opposite is observed.

The results on CIFAR-10 further demonstrate the scalability of residual learning: a 110-layer ResNet achieves 6.43% error, and training converges successfully even for a 1202-layer network [4]. A modest degradation in test error for the 1202-layer model (7.93% vs. 6.43% for ResNet-110) is attributed to overfitting on the small dataset rather than any optimization failure, suggesting that stronger regularization strategies could recover the remaining gap [4].

4. References

- [1] Bernhard E. Boser, Isabelle M. Guyon, and Vladimir N. Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory (COLT '92)*, pages 144–152, Pittsburgh, PA, USA, 1992. Association for Computing Machinery.
- [2] Corinna Cortes and Vladimir N. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.
- [3] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, Cambridge, 2020.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] Brian W. Matthews. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta*, 405(2):442–451, 1975.
- [6] Andrew Ng. Cs229 lecture notes, part v: Support vector machines. <http://cs229.stanford.edu/notes/cs229-notes3.pdf>, 2012. Lecture notes.
- [7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Support vector machines. scikit-learn documentation, 2011. Accessed 2026.
- [8] scikit-learn developers. Breast cancer wisconsin dataset loader. `sklearn.datasets.load_breast_cancer`. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html.
- [9] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.
- [10] Christian Szegedy et al. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [11] William H. Wolberg, Olvi L. Mangasarian, William N. Street, and Nick Street. Breast cancer wisconsin diagnostic dataset. UCI Machine Learning Repository, 1995.
- [12] Doruk Üstündağ. math338_svm_cancer_data: Svm-based breast cancer classification. GitHub repository, 2026.